

DATASMITH AI ASSIGNMENT

PROJECT TITLE:

Multimodal Agentic Application

OVERVIEW:

This project is a multimodal agentic workflow assistant that processes text, images, PDFs and audio files through a unified orchestration pipeline. System detects user intent, selects the appropriate processing workflow and generates structured text-based outputs such as summaries, sentiment analysis, code explanations and action-item extraction.

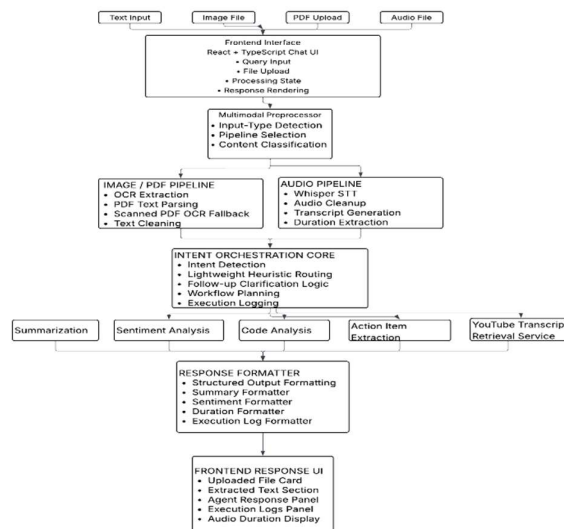
The application supports OCR, PDF parsing with fallback handling, audio transcription, follow-up clarification for ambiguous queries and modular task routing through a lightweight orchestration layer.

SYSTEM ARCHITECTURE:

The platform follows a modular workflow architecture:

1. User input is received through the React frontend
2. FastAPI backend validates and routes requests
3. Input type is automatically detected
4. Appropriate preprocessing pipeline is selected:
 - OCR for images
 - PDF extraction for documents
 - Whisper transcription for audio
5. Intent orchestration core determines the required task
6. Specialized services generate structured outputs
7. Responses are formatted and rendered in the frontend UI

ARCHITECTURE DIAGRAM:



CORE FEATURES:

- Image OCR using Tesseract
- PDF text extraction with OCR fallback
- Audio transcription using Whisper
- Intent-based task orchestration
- Structured summarization pipeline
- Sentiment analysis engine
- Code explanation and complexity analysis
- YouTube transcript retrieval
- Execution logging system
- Multimodal frontend interface

TECH STACK:

CATEGORY	TECHNOLOGIES
Frontend	React, TypeScript, TailwindCSS
Backend	FastAPI, Python
AI/Processing	Tesseract OCR, Whisper Speech-to-Text, PyPDF, Heuristic-based Intent Routing
Utilities	Uvicorn, REST APIs

PROJECT WORKFLOW:

Step 1 - User Input

The workflow begins when the user uploads an image, PDF, audio file, YouTube URL or text query through the frontend interface.

Step 2 - Input Processing

The system identifies the uploaded input type and routes it to the appropriate processing pipeline. Image and PDF files undergo OCR extraction, PDF text parsing, scanned PDF fallback handling and text cleaning. Audio files are processed using Whisper speech-to-text transcription along with audio cleanup, transcript generation and duration extraction. YouTube links are processed through transcript retrieval and transcript normalization.

Step 3 - Intent Detection & Routing

After extraction, the Intent Orchestration Core analyses the processed content to determine the requested operation. Lightweight heuristic-based routing is used to classify the intent and dynamically select the correct workflow.

Step 4 - AI Processing Services

Based on the detected intent, the content is forwarded to specialized AI services such as summarization, sentiment analysis, code analysis and action-item extraction. Each service processes the input independently and generates structured outputs.

Step 5 - Response Formatting

The generated outputs are passed through the response formatting layer, where summaries, sentiment labels, execution logs, duration details and structured response formatting are applied to improve readability and consistency.

Step 6 - Frontend Rendering

Finally, the processed response is displayed on the frontend interface, where users can view uploaded file details, extracted text, AI-generated responses, execution logs and additional metadata in a clean structured UI.

USE CASES:

- Extracting and summarizing scanned PDFs
- Audio lecture transcription and summarization
- OCR-based code explanation from screenshots
- Sentiment analysis for text inputs
- Meeting-note action item extraction
- YouTube transcript summarization

LIMITATIONS:

Current limitations include the following:

- Code analysis currently uses lightweight heuristic-based reasoning
- OCR accuracy may vary depending on image quality
- YouTube transcript extraction depends on transcript availability
- Audio transcription accuracy may reduce for noisy recordings

FUTURE IMPROVEMENTS:

Below listed features can enhance the project in future:

- Advanced LLM-powered reasoning
- True multi-agent execution planning
- Vector database integration
- Streaming responses
- Better multilingual transcription support
- Cost estimation layer

PROJECT SETUP:

Setup Instructions

1. Open Project Folder

Open the project folder in VS Code or terminal.

2. Create Virtual Environment

```
python -m venv venv
```

Activate environment:

```
venv\Scripts\activate
```

3. Install Backend Dependencies

```
pip install -r requirements.txt
```

4. Install Frontend Dependencies

```
npm install
```

5. Run FastAPI Backend

```
uvicorn main:app --reload
```

Backend runs at:

<http://127.0.0.1:8000>

6. Run Frontend

```
npm run dev
```

Frontend runs at:

<http://localhost:5173>

7. Configure OCR Dependencies

Tesseract OCR is required for image text extraction

Poppler is required for scanned PDF OCR fallback support

Both dependencies should be added to the system PATH before running the application

8. Supported Input Types

- Text Queries
- Image Files (.png, .jpg, .jpeg)
- PDF Documents
- Audio Files (.mp3, .wav, .m4a)

9. Output Format

All outputs are generated as structured text responses and displayed through the frontend response interface.

SCREENSHOTS:

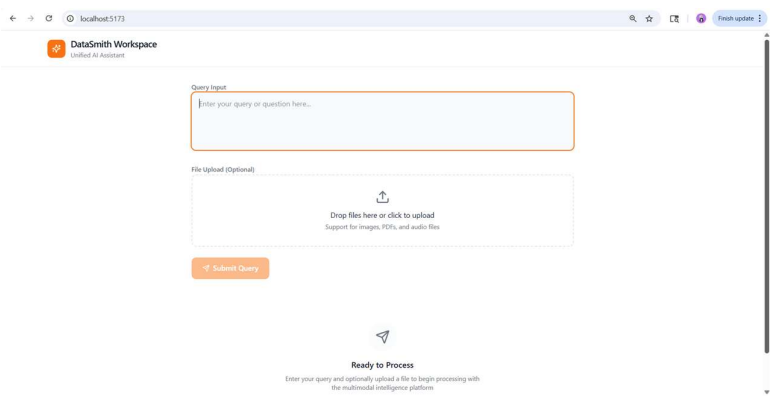


Figure 1: Main multimodal workflow interface

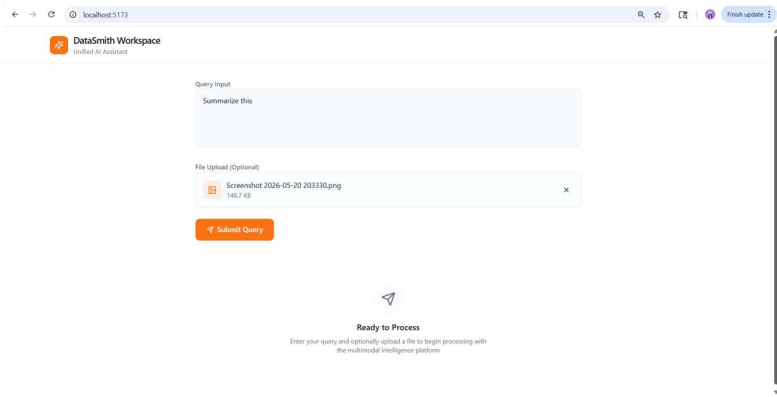
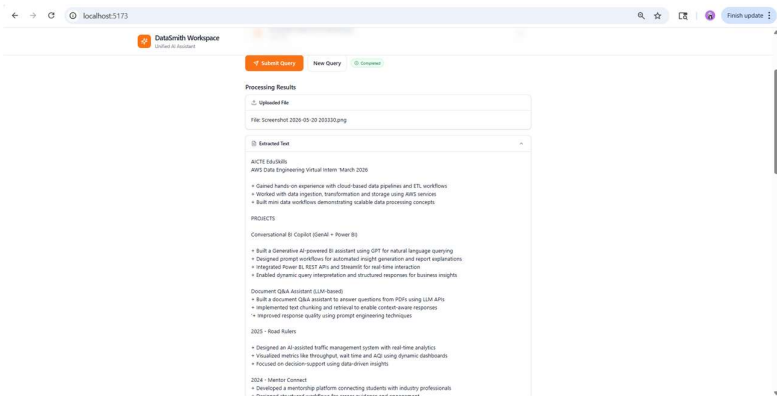


Figure 2.1: Uploading of files

OCR Based Image Screening and Summarization:



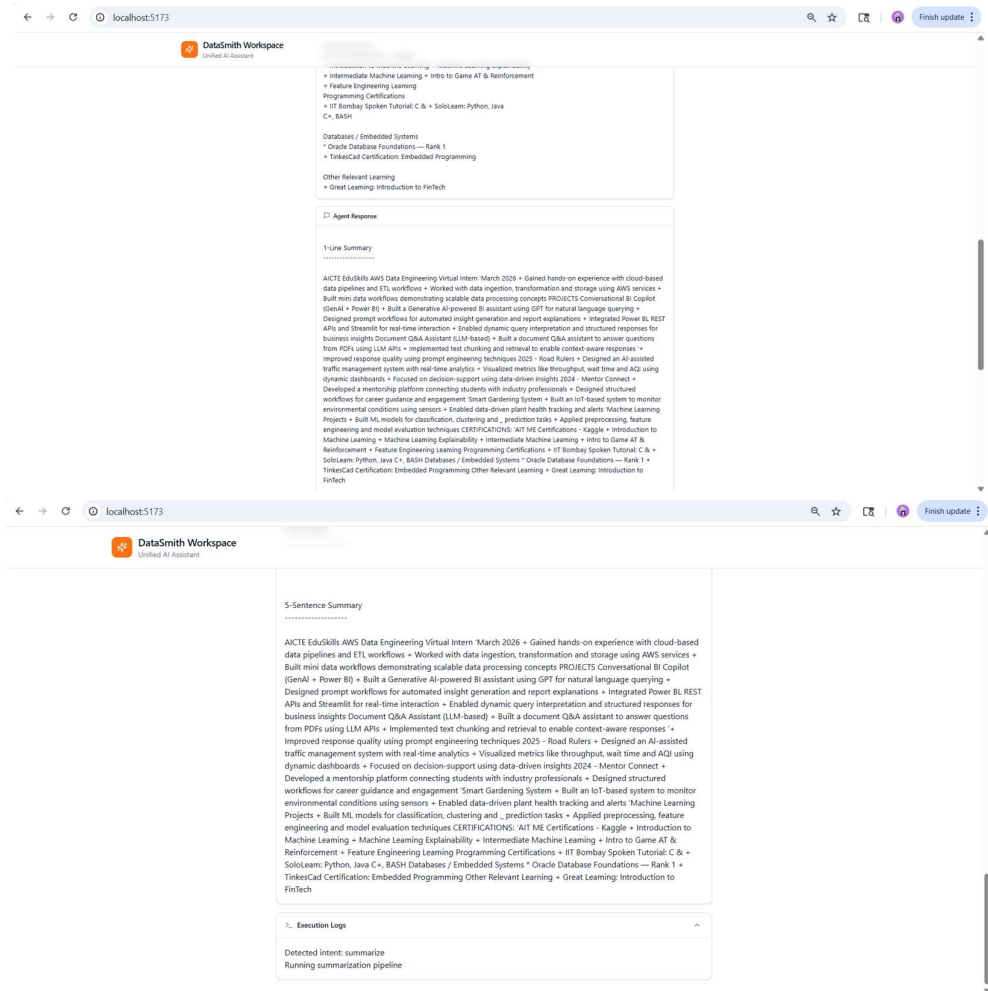
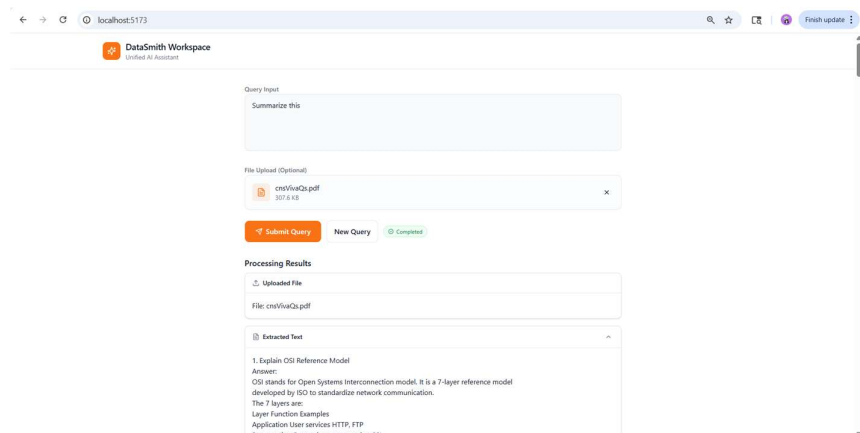


Figure 2.2: OCR-based image extraction and summarization workflow

PDF processing and structured response generation:



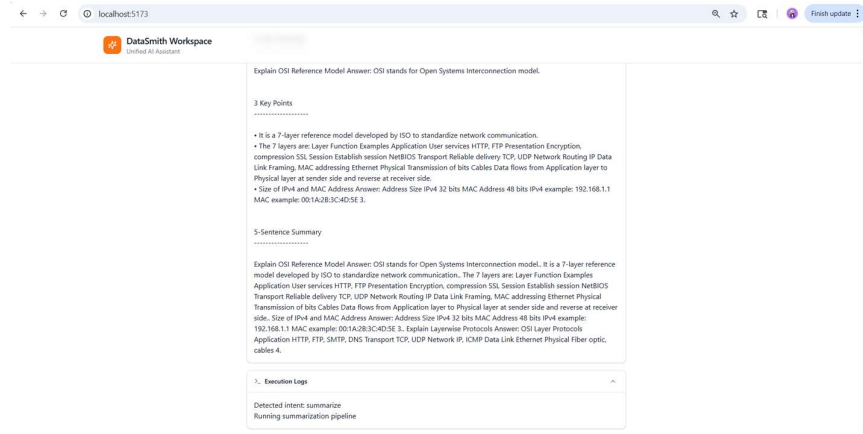


Figure 3.1: Summary of PDF

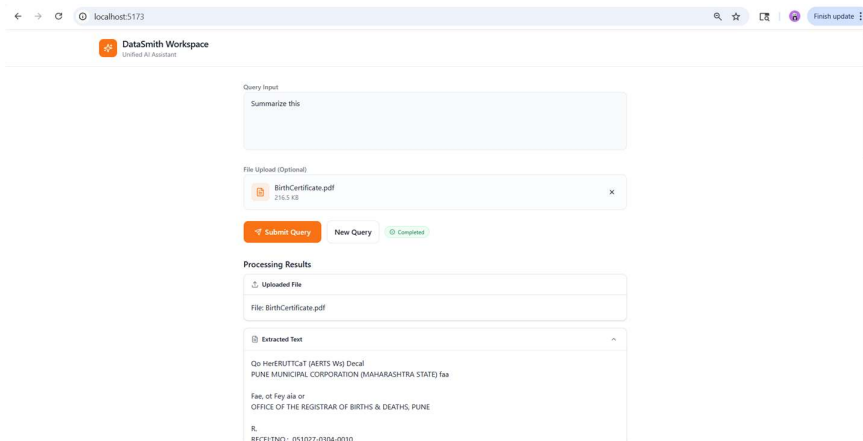


Figure 3.2: Summary of scanned PDF

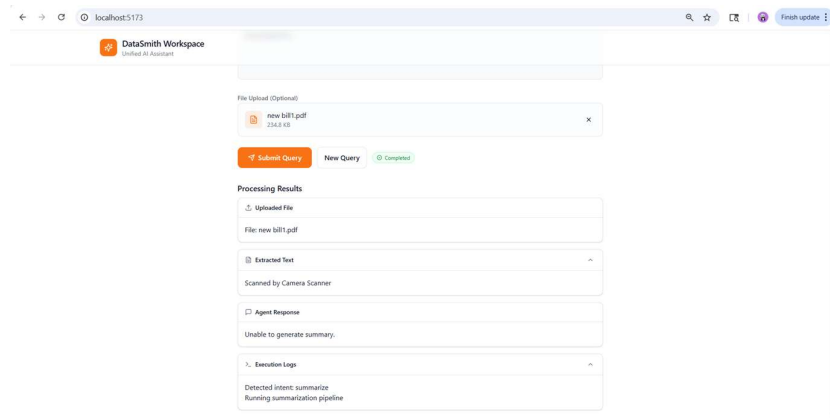


Figure 3.3: Handwritten scanned PDF uploaded

Audio Transcription:

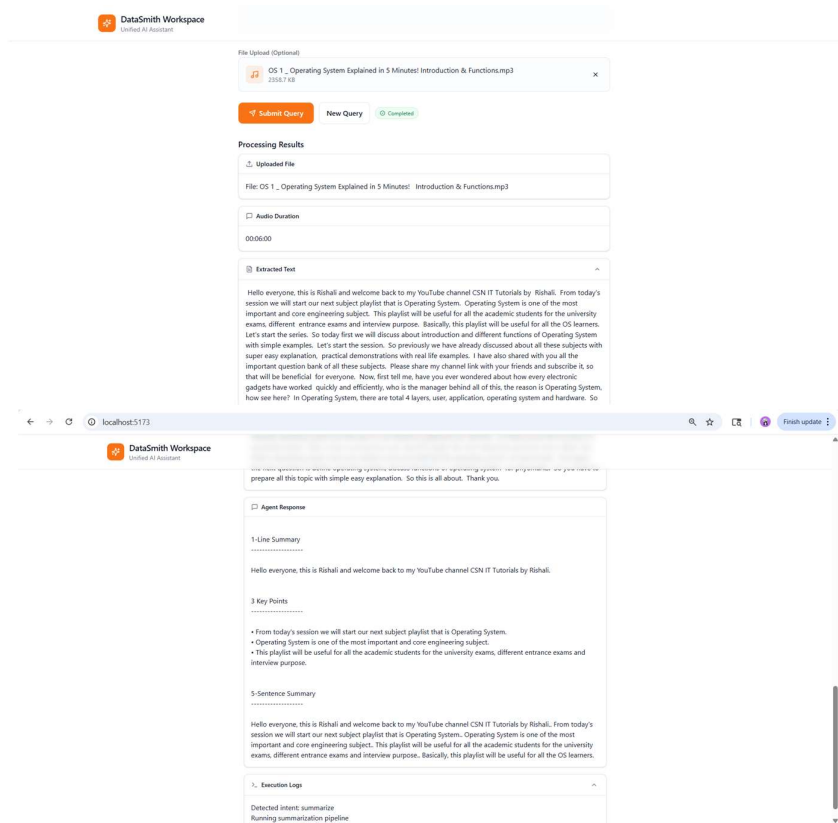


Figure 4.1: 6-minute audio transcription

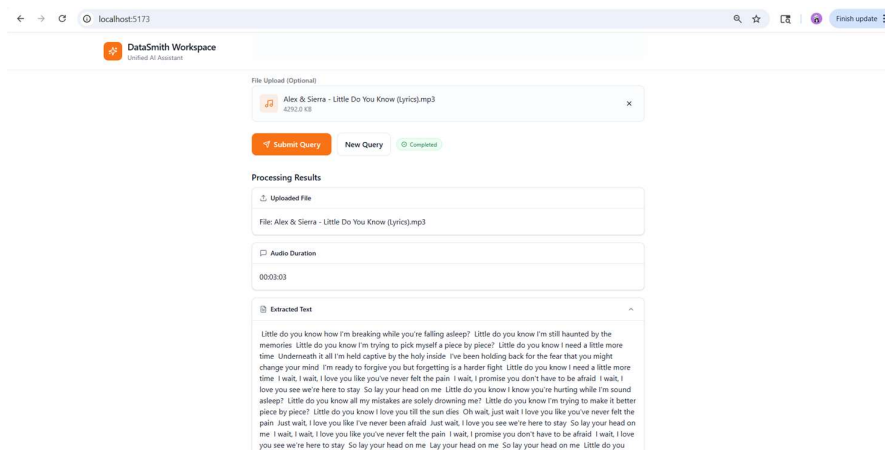


Figure 4.2: Another audio (song) transcription

OCR-based code analysis and complexity estimation:

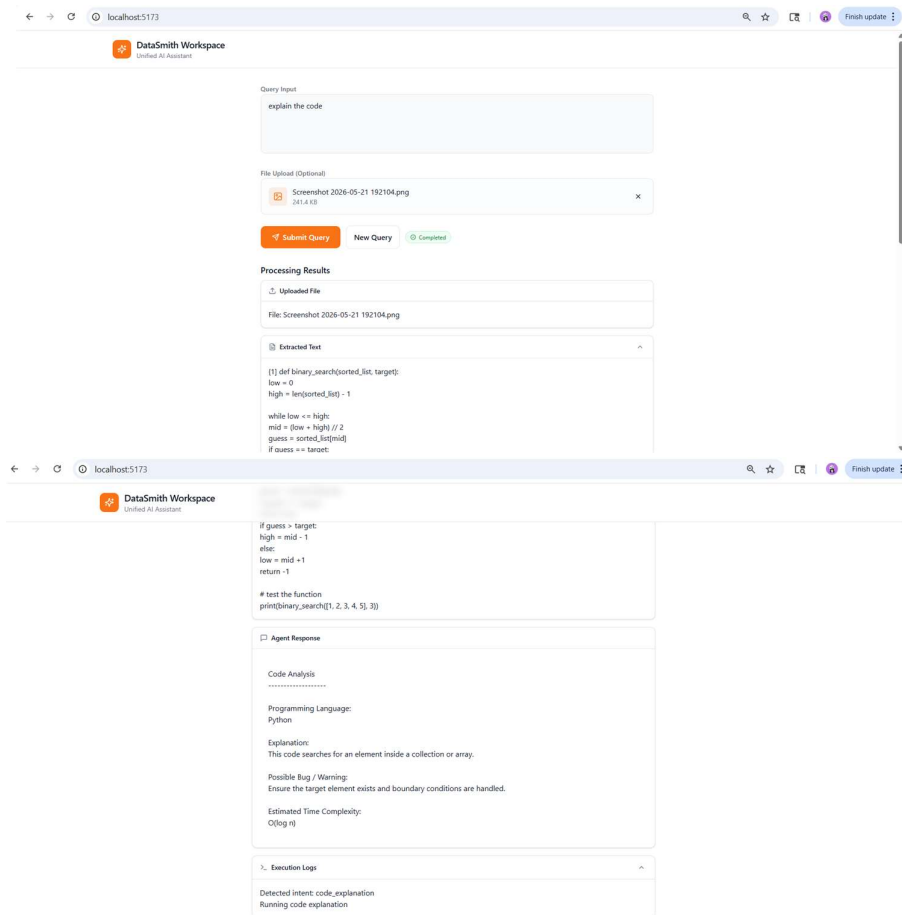
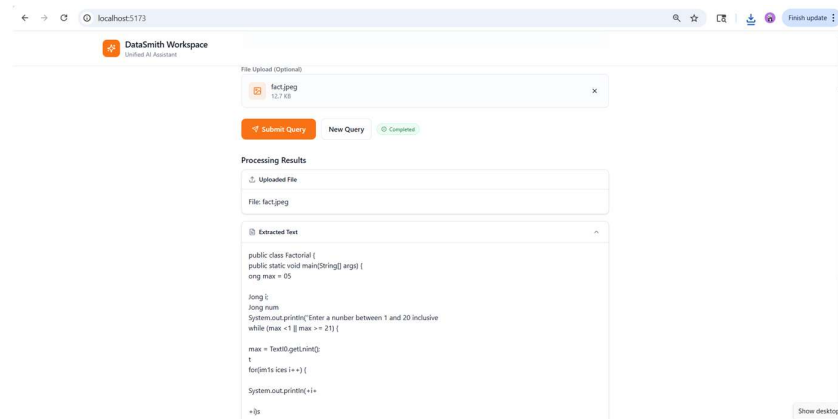


Figure 5.1: Searching code in Python



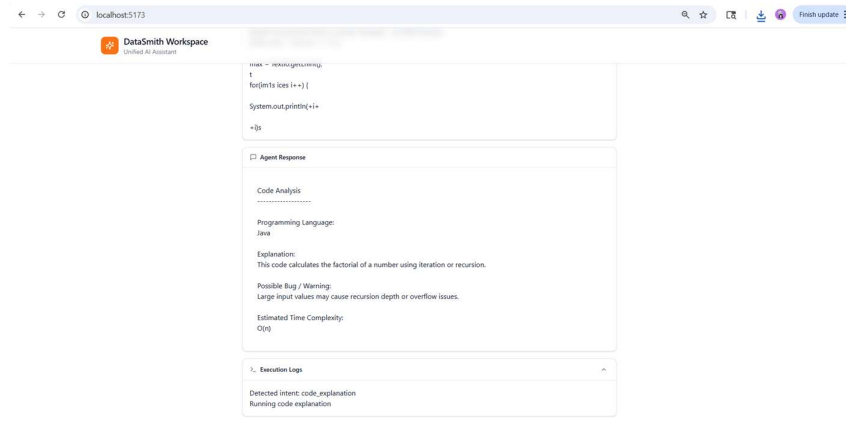


Figure 5.2: Factorial code in Java

Sentiment Analysis Response Generation:

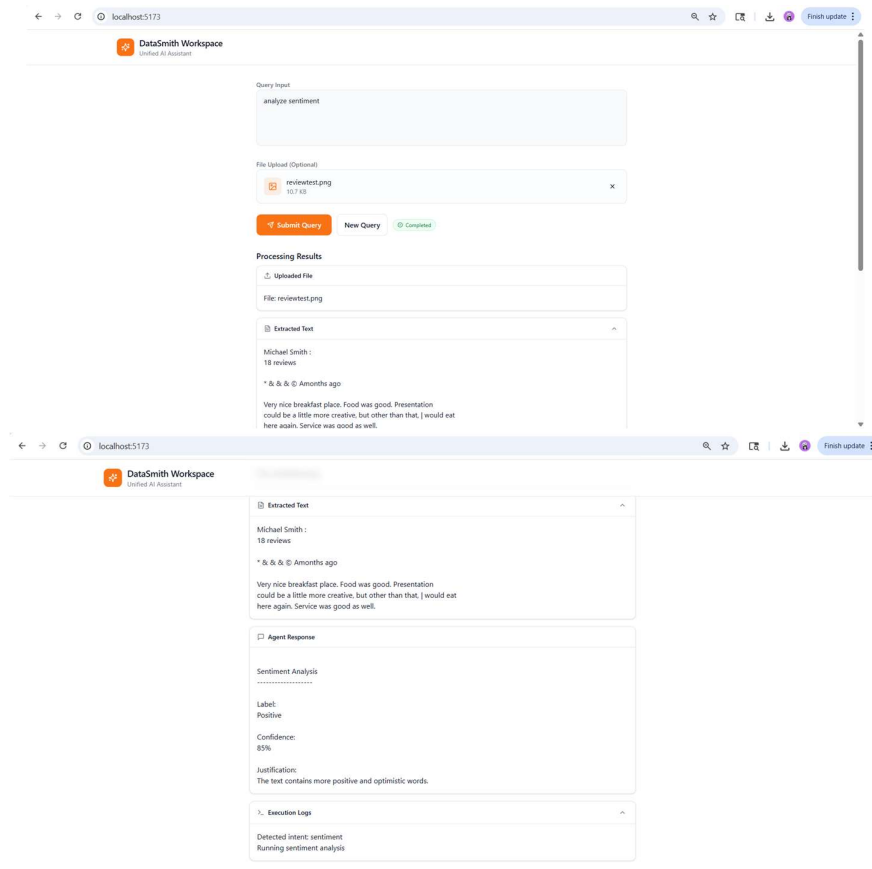


Figure 6.1: Positive result in sentiment analysis

Action-item Extraction:

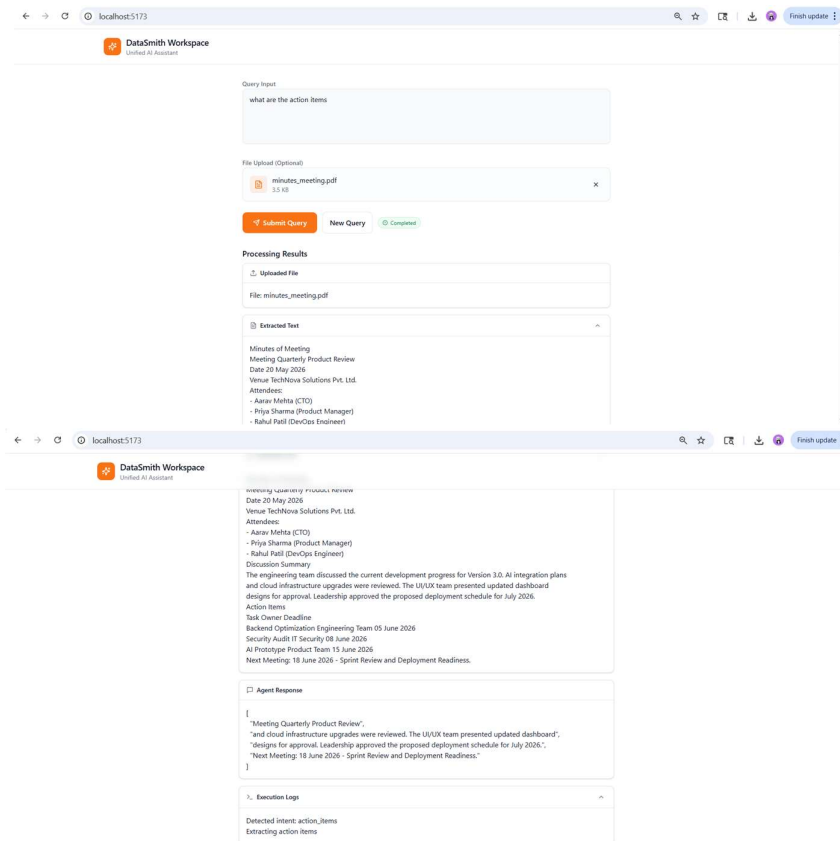


Figure 7.1: Action-item extraction from uploaded meeting-notes PDF

FOLLOW-UP CLARIFICATION:

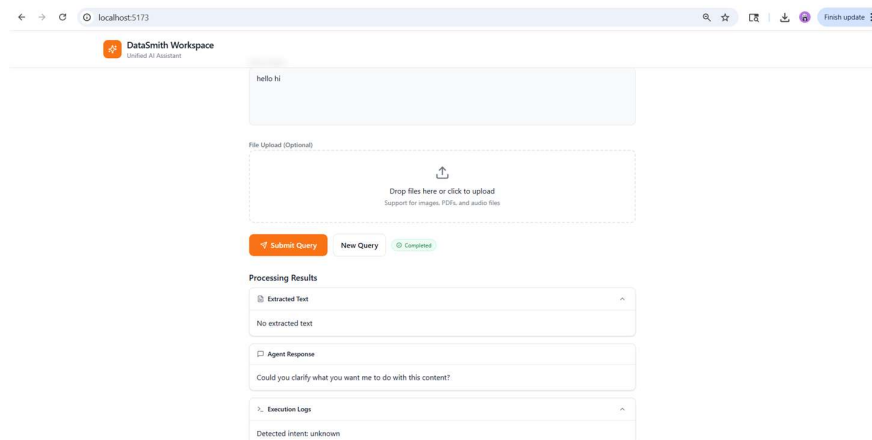


Figure 8.1: Follow-up clarification handling for ambiguous user intent

AUTHOR:

Michelle Vinod

Email ID: michellevinod1609@gmail.com

LinkedIn: www.linkedin.com/in/michellevinod